

CHASE-VLA: Post-Training Quantization Framework for Vision-Language-Action Models with Chunk-Aware Scale Estimation

Jin Hyun^{1,2}, Jung Gyu Min^{1,2}, Gyuhyun Jung^{1,2}, and Youngjoo Lee¹

¹ KAIST, Daejeon, South Korea
youngjoo@kaist.ac.kr

² POSTECH, Pohang, South Korea
{hyun, mjg1104, jgh1020}@postech.ac.kr

Abstract. Vision-Language-Action (VLA) models map visual observations and language instructions to continuous robot actions, but a diffusion-based action expert (AE) poses a key challenge for low-bit post-training quantization (PTQ). The AE is repeatedly invoked across denoising steps and policy queries, where fixed calibration scales can be mismatched with activation ranges that vary with denoising progress and intended motion. We propose CHASE-VLA, a chunk-aware PTQ method that exploits a VLA-specific signal readily available from the policy: the generated action chunk, including its unexecuted future suffix. Rather than relying only on static scale matching for AE layers, CHASE-VLA combines the previously generated chunk as causal action context with denoising step group information to adapt AE activation scales. This enables W4A4 quantization of both MLP and attention projections in the repeated AE without modifying the pretrained policy. On LIBERO, CHASE-VLA achieves 97.3% average success rate on $\pi_{0.5}$ when both MLP and attention projections in the AE are quantized to W4A4, restoring FP16-level performance. CHASE-VLA also reduces the weight storage of the quantized AE linear layers by 73.4% and their single-chunk memory traffic by 70.9% and 71.2% on $\pi_{0.5}$ and GR00T N1.6, respectively, with a predictor overhead of at most 1.26% of the saved storage.

Keywords: Vision-Language-Action model · Post-training quantization

1 Introduction

Vision-Language-Action (VLA) models have emerged as a foundation for embodied intelligence by connecting vision-language understanding with executable robot control [2, 4–6, 14, 15]. By integrating visual perception, language understanding, high-level reasoning, and action generation within a single policy, these models allow robots to interpret natural language instructions, ground them in visual observations, and produce control commands for manipulation tasks [4, 5, 11, 13–15]. Previous VLA systems typically treat action generation as

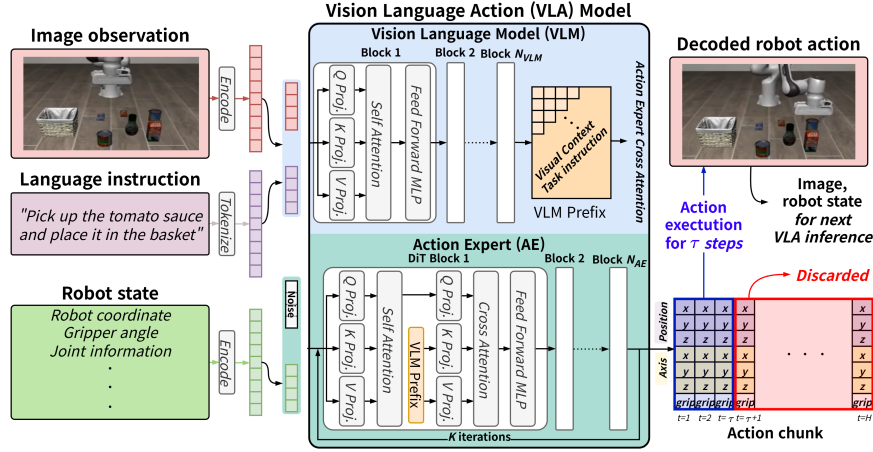


Fig. 1: VLA architecture with a diffusion-based AE for action generation.

a lightweight output interface, where robot actions are discretized into language-compatible tokens and predicted through the Vision-Language Model (VLM) output head rather than a dedicated action-generation module [5–7, 14, 23].

In contrast, recent VLA architectures increasingly preserve the perception and reasoning capability of the VLM branch while introducing a specialized diffusion-based action expert (AE) for continuous trajectory generation [4, 9, 11, 15, 19, 24, 26]. This transition makes the AE a central component of modern VLA policies, because it directly determines trajectory smoothness, temporal consistency, and control robustness during robot execution [4, 15, 24, 30, 31, 33]. Fig. 1 summarizes this architectural trend: the VLM branch provides vision-language grounding, while a separate diffusion-based AE repeatedly refines continuous action chunks for robot control.

Unlike action-token decoders, a diffusion-based AE generates action chunks through iterative refinement, repeatedly processing transformer blocks across denoising steps [9, 15, 21]. This computation recurs at every replanning step throughout a rollout, so weight reads and activation movement accumulate over the entire trajectory [33]. The AE is therefore a primary target for deployment-oriented quantization because it directly affects both memory traffic and control fidelity in repeated action generation [31]. Among compression options, post-training quantization (PTQ) is a practical approach for pretrained policies because it avoids the robot data collection, rollout validation, and iterative action generation required by quantization-aware training [12, 20, 22]. This makes low-bit PTQ attractive for deployment, but the quantized AE must still preserve continuous-control precision across repeated denoising and replanning throughout the executed trajectory [31, 33].

Low-bit PTQ for the AE therefore differs from language-model quantization in both sensitivity and execution pattern [1, 10, 17, 28, 33]. Previous VLA PTQ methods keep sensitive attention projections in floating point and quantize

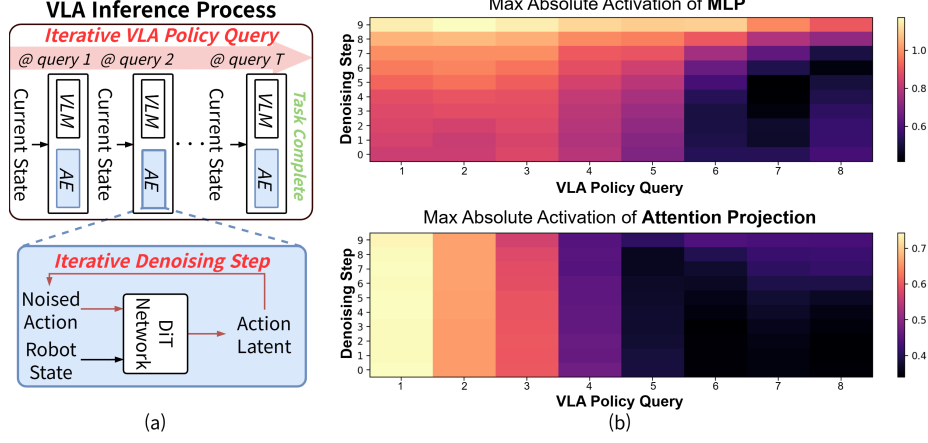


Fig. 2: Activation-scale variation in a diffusion-based VLA AE. (a) Repeated AE calls across denoising steps and policy queries. (b) Maximum absolute activation values across denoising steps and policy queries, which make fixed calibration scales prone to clipping or loss of quantization resolution.

only the MLP projections [30, 33], leaving a large fraction of the repeated AE cost unquantized. This limitation stems from a mismatch between static calibration scales and the dynamic activation behavior of the repeatedly executed AE [8, 27, 33, 35]. As illustrated in Fig. 2(a), the AE is invoked in two nested loops: it refines each action chunk over multiple denoising steps and regenerates a new chunk at every replanning step throughout a rollout [4, 9, 15]. Across denoising steps, the AE produces physical control values through iterative refinement, so its activations change with the denoising step rather than following a fixed forward-pass distribution [33]. The activation range also changes with the action context. Each policy query is conditioned on a different observation, instruction, and robot state, so the generated motion, and therefore the required activation scale, can vary substantially from one chunk to the next [4, 15, 30]. Fig. 2(b) shows this behavior within individual AE linear layers, where the maximum absolute activation varies across both denoising steps and action contexts. A single calibration-derived scale cannot accommodate these variations without either clipping broad activations or losing resolution for fine control [33, 36]. Across repeated denoising and replanning, the resulting local errors can accumulate into trajectory-level control errors [30]. Addressing this mismatch requires activation scales that depend on both the denoising step and the motion being generated [8, 27, 36]. However, the action context of the current chunk is not causally available before the AE produces that chunk, making direct current-chunk scale selection impractical.

Chunk-based VLA execution provides a causal source of action context for resolving this causality constraint. Modern VLA policies predict short action chunks at each policy query instead of a single immediate action [2, 4, 9, 15].

Each chunk constitutes a short-horizon plan conditioned on the current observation, language instruction, and robot state. As illustrated in Fig. 1, the robot executes only an initial prefix of the chunk and then replans after receiving a new observation [2, 4, 9]. The remaining suffix is not executed because the next policy query generates a new chunk based on the updated observation. However, this suffix is still the policy’s predicted continuation of the same trajectory whose prefix is executed. Thus, it provides an accessible signal of the intended near-future motion without additional rollouts or task-specific annotations.

We propose CHASE-VLA, a chunk-aware PTQ method for low-bit quantization of diffusion-based VLA AEs. Instead of using fixed calibration scales, CHASE-VLA modulates calibrated activation scales with action-conditioned factors that are shared within groups of denoising steps. The conditioning signal is the previously generated action chunk, which compactly summarizes the intended near-future motion and enables scale selection to reflect both action context and denoising progress without modifying the AE architecture. Through adaptive scaling, CHASE-VLA supports 4-bit weight and activation quantization (W4A4) and mitigates the scale mismatch that typically arises from static PTQ. Unlike previous VLA PTQ methods, which keep sensitive attention projections in floating point for stability, CHASE-VLA extends W4A4 coverage beyond MLP layers to the attention projection layers within the repeated AE. By including these repeatedly invoked projections in the low-bit quantization target, CHASE-VLA achieves further reductions in memory traffic compared to MLP-only AE quantization.

Our contributions are summarized as follows.

- We identify activation-scale mismatch across denoising steps and action contexts as a central barrier to low-bit PTQ for diffusion-based VLA AEs.
- CHASE-VLA estimates activation scales from the previous action chunk and applies them at the denoising step group level.
- CHASE-VLA expands the W4A4 AE target from MLP projections to both MLP and attention projections while keeping the pretrained policy and the AE architecture unchanged.
- Experiments on LIBERO across $\pi_{0.5}$, GR00T N1.6, and CogACT show that this design largely preserves task success under the expanded W4A4 target, while an efficiency analysis on $\pi_{0.5}$ and GR00T N1.6 quantifies reductions in quantized AE linear weights and single-chunk memory traffic with modest predictor overhead.

2 Related Work

2.1 Vision-Language-Action Models

Recent VLA research has shifted from discrete action prediction to continuous trajectories generated by diffusion-based AEs. Diffusion Policy demonstrated that modeling robot actions as a conditional denoising process is effective for generating smooth and multimodal action trajectories [9]. Following this direction,

recent systems such as CogACT, $\pi_{0.5}$, GR00T, and RDT employ diffusion-based action generation modules to couple high-level vision-language reasoning with low-level continuous control [2–4, 15, 19]. These AE architectures are attractive because they predict action chunks rather than isolated actions, allowing temporally correlated future controls to be generated as a coherent trajectory [2, 3, 9].

The chunk-based formulation enhances trajectory smoothness and execution reliability, which are critical for contact-rich manipulation and long-horizon robotic tasks [2, 9, 15, 19]. However, this property introduces a substantial deployment burden as the AE repeatedly evaluates action chunks during multiple denoising steps. The AE is therefore a major determinant of continuous-control quality and a key target for low-bit deployment [30, 33].

2.2 Efficient VLA Inference and Compression

Recent studies have explored several approaches to reducing the inference cost of VLA models without sacrificing their embodied control capabilities. TinyVLA combines compact multimodal transformers with lightweight diffusion-policy heads, while EfficientVLA accelerates inference by pruning redundant language layers and reusing intermediate representations [25, 32]. VLA-Cache further improves runtime efficiency by reusing cached key-value features for static visual tokens, and MoLe-VLA dynamically skips non-essential computation through mixture-of-layers routing in the language backbone [29, 34]. Most of these efforts primarily modify the model architecture or inference execution path, while the numerical precision of VLA policies remains less explored. For lightweight deployment under tight memory and bandwidth budgets, this leaves quantization as an important requirement beyond architectural or execution-level optimization. This requirement is particularly relevant to diffusion-based AEs, where low-bit inference must preserve action-chunk-dependent activation distributions and denoising step dynamics.

2.3 Post-Training Quantization for LLMs, VLMs, DiTs, and VLAs

For VLA AE quantization, a central distinction is not only quantization scope but also the information used for activation-scale selection during iterative action generation. Table 1 organizes this distinction across relevant PTQ categories and highlights the remaining gap addressed by CHASE-VLA.

PTQ reduces the memory footprint and inference cost of pretrained transformers without full retraining [10, 28]. For large language and vision-language models, representative methods such as SmoothQuant, GPTQ, and AWQ address low-bit deployment by reducing activation outliers, compensating weight quantization error, or redistributing activation energy through smoothing and scaling [1, 10, 17, 28]. These methods provide useful building blocks for quantizing transformer backbones in VLA systems, where semantic grounding and multimodal reasoning are handled by the VLM branch. However, their scale choices are mainly designed for token-generating transformer inference and do not explicitly model the iterative denoising dynamics or action-context variation

Table 1: Scale-selection signals and quantization scopes across PTQ categories.

PTQ Category	Scale-Selection Signal	Quantization Scope
LLM/VLM PTQ	Weight/activation statistics	Transformer encoder/decoder
Diffusion PTQ	Denoising step signal	Diffusion denoising blocks
VLA PTQ	Static module calibration	AE MLP only
CHASE-VLA	Previous action chunk	AE MLP
(Proposed)	+ denoising step group	+ attention projections

of diffusion-based AEs. Directly transferring these assumptions to VLA action generation can therefore be insufficient when quantization errors perturb continuous robot commands.

Quantization has also been studied for diffusion models and diffusion transformers (DiT), whose activation statistics vary across denoising steps [8, 27]. Methods such as PTQ4DiT and Q-DiT improve diffusion quantization through denoising-step-aware calibration, fine-grained grouping, dynamic quantization, or low-rank residual compensation. These methods are relevant to modern VLA policies because diffusion-based AEs also refine outputs over denoising steps. Nevertheless, diffusion PTQ methods are primarily developed for image or video generation, where scale adaptation is usually tied to the denoising process itself. They do not use the predicted action chunk as causal context for selecting activation scales in subsequent AE inference. This leaves a gap between denoising-step-aware diffusion quantization and action-context-aware quantization for VLA AEs.

More recently, VLA-specific PTQ methods have begun to address the numerical sensitivity of VLA pipelines directly. QuantVLA analyzes scale drift in DiT-based VLA AEs and introduces calibration-based stabilization for VLA PTQ [33]. Its AE quantization uses a selective target: attention projections are kept in floating point while the low-bit target focuses on MLP projections [33]. DA-PTQ addresses a complementary cross-space mismatch by aligning the conditioning representation passed from the vision-language backbone to the action decoder [30]. This conditioning-pathway alignment is distinct from adapting activation scales inside the repeatedly executed AE.

Together, these VLA-specific methods improve PTQ stability but still leave open the issue of W4A4 quantization across both MLP and attention projections within the AE. Rather than relying only on static calibration statistics, denoising step cues, or cross-module representation alignment, CHASE-VLA uses the previously generated action chunk and denoising step groups as conditioning signals for activation-scale estimation. This formulation directly targets the action-context- and denoising-step-dependent scale mismatch that arises during iterative action chunk generation.

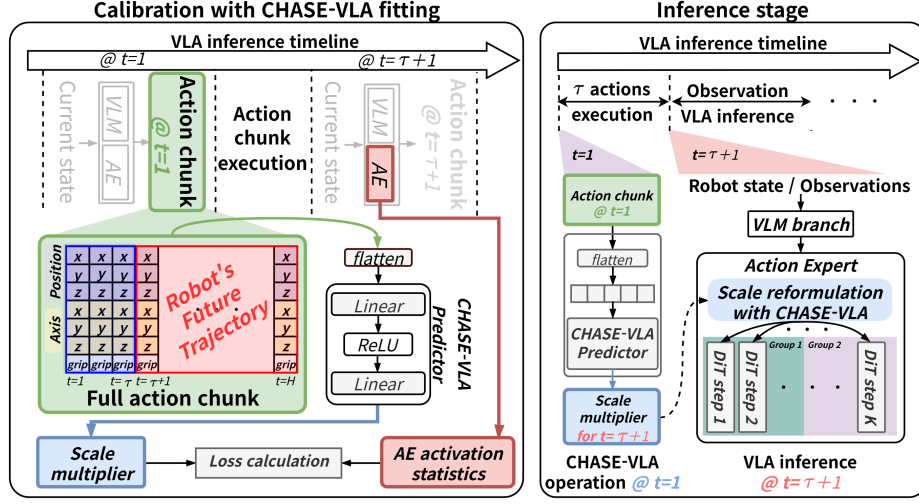


Fig. 3: Overview of CHASE-VLA. After each policy query, the generated action chunk, including its unexecuted future suffix, provides causal action context for precomputing activation scales for denoising step groups in the next query.

3 Method

3.1 Preliminaries and Static PTQ

Modern VLA policies commonly pair a vision-language branch with a diffusion-based AE. Let t denote a control time at which the policy is queried, and let $\tau \leq H$ be the number of actions executed before the next query. Given an observation image I_t and a language instruction L , the vision-language branch produces a conditioning representation $F_t = \text{VLM}(I_t, L)$. Together with the robot proprioceptive state r_t , this representation conditions an AE that iteratively refines an action latent and decodes it into an action chunk $A_t = [a_{t,0}, \dots, a_{t,H-1}]$, where $a_{t,h} \in \mathbb{R}^{d_a}$ denotes the h -th robot action in the chunk, d_a is the action dimension, and H is the action horizon. At the AE denoising step $k \in \{0, \dots, K-1\}$, where K is the total number of denoising steps, let x_k denote the action latent at step k . The AE update is

$$x_k = f_\theta(x_{k+1}, F_t, r_t, k), \quad (1)$$

where f_θ is the denoising function. The final denoised latent is mapped to the executable chunk by the action decoder,

$$A_t = \text{Dec}(x_0). \quad (2)$$

Dec denotes the final action decoder.

CHASE-VLA targets this repeated action-generation path. Each target linear layer in f_θ is invoked across denoising steps and replanned chunks, so its memory

traffic is repeatedly amplified during a rollout. At the same time, numerical errors in this path directly perturb continuous robot actions rather than only semantic token probabilities.

For a target linear layer l , let $u_{t,l,k}$ denote its input activation tensor when the policy generates chunk A_t at denoising step k . Uniform PTQ quantizes this tensor as

$$\begin{aligned} q(u; s) &= \text{clip}(\text{round}(u/s), q_{\min}, q_{\max}), \\ \hat{u}(s) &= q(u; s) \cdot s. \end{aligned} \tag{3}$$

In this expression, $q(u; s)$ is the integer quantization of activation tensor u , $\hat{u}(s)$ is its dequantized value, s denotes the activation scale, $q_{\max} = 2^{a_{\text{bits}}-1} - 1$ and $q_{\min} = -q_{\max}$ for symmetric signed a_{bits} -bit activations, and the operator is applied element-wise. The scale s determines the symmetric clipping range.

Static PTQ derives the activation scale of each target layer from a calibration set and keeps it fixed throughout inference. In Eq. 3, a fixed scale s sets the clipping threshold to $q_{\max} \cdot s$. If the actual activation range exceeds this threshold, activations are clipped; if it is much smaller, the quantization step becomes unnecessarily coarse, reducing resolution. However, this range varies with the denoising step and the policy query, since each query generates a different action chunk. A fixed scale, therefore, cannot track the per-query and per-step activation range, motivating an activation scale that depends on both the denoising step and the action context.

3.2 Chunk-Aware Scale Estimation

To address this variation, activation-scale selection should consider both the denoising step and the specific action being generated. A fixed calibration scale ignores both factors, while a denoising-step-only rule still assigns identical scales to different action chunks processed at the same denoising step. As illustrated in Fig. 3, CHASE-VLA overcomes these limitations by retaining the quantization operator in Eq. 3 while reusing the previously generated action chunk to precompute group-wise activation scales for generating the next action chunk. Instead of predicting an absolute scale independently, CHASE-VLA predicts a multiplicative adjustment to a calibration-derived base scale, utilizing the generated action context and the denoising step group.

CHASE-VLA derives action context from action chunks previously predicted by the policy. For a policy query at time t , the preceding query at $t - \tau$ generated a full action chunk $A_{t-\tau}$, and the robot executed only its initial τ actions before replanning. Although the remaining suffix is unexecuted, it represents the policy’s predicted continuation of the executed prefix and thus encodes the intended near-future motion. Therefore, as shown in Fig. 3, CHASE-VLA conditions the activation scale for the current chunk A_t on the entire previous chunk, including both the prefix and the suffix. This approach provides a signal that captures motion magnitude and temporal structure without requiring ground-truth future actions or rollout annotations.

The scale adjustment is also conditioned on the denoising step. Each adjustment is a multiplicative factor on the base scale. Rather than assigning an independent factor to every step, CHASE-VLA maps the K denoising steps to M groups indexed by $m \in \{1, \dots, M\}$ and predicts one factor per group. This grouping keeps the predictor output compact while still distinguishing the differing activation ranges of coarse early and fine late refinements.

CHASE-VLA initially determines the base scale, which subsequently undergoes adjustment based on action-conditioned group parameters. Here, $\tilde{u}_{l,m}$ represents the recorded calibration activations for layer l corresponding to denoising step group m . The group-specific initial scale is

$$s_{l,m}^{\text{init}} = \max\left(\epsilon_s, \frac{\text{absmax}(\tilde{u}_{l,m})}{q_{\max}}\right), \quad (4)$$

where $\text{absmax}(\cdot)$ takes the maximum absolute value over all tensors in the calibration activation set and ϵ_s is a small scale floor. These initial scales are used when generating the first action chunk of an episode, where no previous action chunk is available, and they anchor the relative scale adjustments used for later queries. For a policy query at time t , CHASE-VLA forms the action context by flattening the previously generated chunk,

$$z_t = \text{vec}(A_{t-\tau}) \in \mathbb{R}^{Hd_a}. \quad (5)$$

The conditioning process is causal since z_t is computed prior to the generation of the current chunk A_t by the AE. As shown in Fig. 3, this process establishes a recurrent scale update path. The complete chunk generated at time $t - \tau$, including its unexecuted future suffix, is reused as context for selecting activation scales at time t .

For each target linear layer l , f_{ϕ_l} denotes the lightweight CHASE-VLA predictor shown in Fig. 3. This predictor maps the normalized action context to one log-scale adjustment and one uncertainty estimate per denoising step group:

$$[\hat{y}_{t,l,1:M}, \log \hat{\sigma}_{t,l,1:M}^2] = f_{\phi_l}(\text{norm}(z_t)). \quad (6)$$

The output $\hat{y}_{t,l,m}$ is the predicted log-scale adjustment for group m , $\log \hat{\sigma}_{t,l,m}^2$ is the predicted log-variance used for uncertainty estimation, and $\text{norm}(\cdot)$ denotes feature standardization with calibration-set mean and standard deviation. For denoising step group m , CHASE-VLA converts the predictor output into an uncertainty-aware scale multiplier,

$$\gamma_{t,l,m} = 10^{\hat{y}_{t,l,m} + \alpha \hat{\sigma}_{t,l,m}^2}, \quad (7)$$

where $\hat{\sigma}_{t,l,m} = \exp(0.5 \log \hat{\sigma}_{t,l,m}^2)$ and α is a fixed margin. The uncertainty term moderates the context-specific adjustment when the prediction is uncertain, reducing the risk of an overly aggressive scale reduction under action contexts that deviate from the calibration distribution. The adaptive factor is shared within

denoising step group m and corresponds to the scale multiplier shown in Fig. 3; it modulates the group-specific initial scale to produce the final group scale:

$$s_{t,l,m}^{\text{CHASE-VLA}} = s_{l,m}^{\text{init}} \cdot \gamma_{t,l,m}. \quad (8)$$

During denoising, each denoising step in the AE utilizes the precomputed scale of its assigned group. Eq. 8 therefore defines the activation scale for the current query from two sources: the previous action chunk $A_{t-\tau}$ and the denoising step group of the current linear call.

To fit the predictor, each collected calibration query is treated as a supervised pair: the previous chunk provides the predictor input, and the recorded activation statistics from producing the corresponding current chunk provide the target. For the i -th collected query at time t_i , the predictor input is the stored context $\text{vec}(A_{t_i-\tau})$, while $\tilde{u}_{t_i,l,m}$ is the activation observed when the AE generated the current chunk A_{t_i} . As shown in Fig. 3, these observed activation statistics are used only to construct offline calibration targets, not during inference. Unlike the query-independent initial scale in Eq. 4, the current-chunk scale is computed separately for each collected query and denoising step group:

$$s_{t_i,l,m}^{\text{chunk}} = \frac{\text{absmax}(\tilde{u}_{t_i,l,m})}{q_{\text{max}}}. \quad (9)$$

With a small numerical constant ϵ , the group target is the base-10 log-ratio between the group-specific current-chunk scale and the group-specific initial scale

$$y_{t_i,l,m}^* = \log_{10} \left(\epsilon + \frac{s_{t_i,l,m}^{\text{chunk}}}{s_{l,m}^{\text{init}}} \right). \quad (10)$$

This target trains CHASE-VLA to use the previous chunk of the i -th collected query to predict how much of the calibration-derived group-specific range is needed for the corresponding current chunk at t_i . The learned adjustment therefore modulates the calibrated group-specific scale to recover quantization resolution when the base range is unnecessarily broad, rather than regressing an absolute activation scale from scratch. The predictor is fitted on collected calibration queries with a lightweight objective combining a pinball loss on the log-scale target, a Gaussian negative log-likelihood term for the log-variance head, and small parameter regularization. A detailed analysis of the loss function utilized for CHASE-VLA predictor is provided in the supplement. Throughout this calibration, the pretrained VLA policy remains frozen, so CHASE-VLA performs post-training quantization rather than policy fine-tuning.

3.3 Quantization Targets and Inference Path

CHASE-VLA focuses W4A4 quantization on repeated linear projections inside the diffusion AE, where the same projections are invoked across denoising steps and policy queries. The adaptive W4A4 target therefore includes feed forward MLP projections and linear projections in the attention modules of the AE.

These layers lie on the repeated denoising path, where activation-scale mismatch directly affects low-bit action generation.

This target coverage is more aggressive than that of previous VLA PTQ methods, which typically quantize only the MLP in the AE while leaving attention projections in floating point. The difference is important because attention projections are also invoked at every denoising step; keeping them in floating point leaves a substantial repeated data-movement cost within the AE. As described in Sec. 3.2, CHASE-VLA adapts the activation scale to the predicted action context and denoising step group, allowing attention projections to remain within the W4A4 AE target instead of being left in floating point.

As shown in Fig. 3, CHASE-VLA acts as a scale update module for the AE during inference. Since the previous chunk is not available for the first policy query, the target linear layers use the initial scales in Eq. 4. Once the AE generates A_t , CHASE-VLA immediately maps $\text{vec}(A_t)$ to group-wise scaling factors and updates the activation scales for all target layers and denoising step groups of the next policy query at time $t + \tau$. Thus, before the AE starts generating $A_{t+\tau}$, the scales $s_{t+\tau,l,m}^{\text{CHASE-VLA}}$ in Eq. 8 have already been fixed. During the next denoising process, each target linear layer only reads the precomputed scale for its denoising step group m . After $A_{t+\tau}$ is produced, CHASE-VLA similarly updates the scales for the following query from $\text{vec}(A_{t+\tau})$. The pretrained policy weights and denoising schedule remain unchanged; only the activation scale used by each target linear layer is updated between policy queries.

4 Experiments

4.1 Experimental Setup

CHASE-VLA is evaluated on three VLA models, $\pi_{0.5}$, GR00T N1.6, and CogACT, which represent diverse AE architectures and model scales [2, 3, 15]. All models are evaluated on the four LIBERO task suites: Spatial, Object, Goal, and Long [18], and we report the task success rate averaged over 20 episodes for each task within a suite.

We compare CHASE-VLA with the FP16 reference, QuantVLA [33], and diffusion PTQ baselines [8, 27] adapted to the AE. All compared PTQ methods use the same calibration set and evaluation protocol within each model. For the diffusion PTQ baselines, we apply channel-wise salience balancing and Spearman’s ρ -guided salience calibration for PTQ4DiT, and automatic granularity allocation with sample-wise dynamic activation quantization for Q-DiT. QuantVLA is implemented by following its calibration and static scale-matching procedure using attention temperature matching and output head balancing. Each evaluation focuses on the AE, while the VLM linear layers are quantized using low-bit PTQ methods with the LLM-oriented PTQ scheme [16], matching the VLM treatment in QuantVLA [33], ensuring that the reported differences are due solely to AE quantization. We also report AE latency, quantized linear layer storage and memory traffic, and predictor overhead.

Table 2: LIBERO success rates on $\pi_{0.5}$.

Method	Precision	AE quant. target		Spa.	Obj.	Goal	Long	Avg.
		MLP	Attn. proj.					
FP16	FP16	–	–	98.5	99.0	97.5	93.5	97.1
QuantVLA	W4A8	✓	–	98.5	98.0	98.0	96.0	97.6
QuantVLA (expanded)	W4A4	✓	✓	98.5	98.5	93.5	90.5	95.3
Q-DiT	W4A4	✓	✓	97.0	97.5	95.0	88.0	94.4
PTQ4DiT	W4A4	✓	✓	97.5	94.5	94.0	88.0	93.5
CHASE-VLA (w/o action)	W4A4	✓	✓	98.5	97.5	93.5	92.5	95.5
CHASE-VLA	W4A4	✓	✓	97.0	99.5	98.0	94.5	97.3

Table 3: LIBERO success rates on additional VLA models.

Model	Method	Precision	Spa.	Obj.	Goal	Long	Avg.
CogACT	FP16	FP16	87.5	98.0	95.5	91.5	93.1
	QuantVLA (expanded)	W4A4	89.5	57.5	73.5	81.0	75.4
	Q-DiT	W4A4	81.0	73.5	83.5	73.5	77.9
	PTQ4DiT	W4A4	87.5	77.5	80.5	73.0	79.6
	CHASE-VLA	W4A4	86.5	97.0	88.0	90.5	90.5
GR00T N1.6	FP16	FP16	99.0	99.5	97.5	96.0	98.0
	QuantVLA (expanded)	W4A4	53.0	55.0	56.0	33.5	49.4
	Q-DiT	W4A4	6.5	69.5	12.0	3.5	22.9
	PTQ4DiT	W4A4	6.0	62.5	9.0	2.5	20.0
	CHASE-VLA	W4A4	90.5	99.5	92.0	84.0	91.5

4.2 Quantized VLA Performance

Table 2 presents the LIBERO evaluation results on $\pi_{0.5}$. QuantVLA is evaluated in its original W4A8 MLP-only configuration and on the same expanded W4A4 AE target as CHASE-VLA. CHASE-VLA (w/o action) removes previous-chunk conditioning while retaining denoising step groups.

With the common W4A4 target, CHASE-VLA achieves 97.3% average success, exceeding expanded QuantVLA, Q-DiT, and PTQ4DiT by 2.0, 2.9, and 3.8 percentage points. Its 95% confidence interval (CI), [95.9, 98.2]%, overlaps the FP16 interval, [95.7, 98.1]%, with similar point estimates. Removing action context yields 95.5%, a 1.8-point reduction, with lower Goal and Long success.

Table 3 extends the evaluation to CogACT and GR00T N1.6, where CHASE-VLA achieves 90.5% and 91.5% success, respectively. Their 95% CIs, [88.3, 92.3]% and [89.4, 93.2]%, lie above those of all compared W4A4 PTQ baselines. The largest separation occurs on GR00T N1.6, where expanded QuantVLA, Q-DiT, and PTQ4DiT reach 49.4%, 22.9%, and 20.0%. Detailed confidence interval results are reported in Supplementary.

Table 4: Ablation study of CHASE-VLA on $\pi_{0.5}$ across the four LIBERO suites. Success rates correspond to the settings in the same order; bold entries mark the default CHASE-VLA configuration. Prefix and suffix denote the executed and unexecuted portions of the previous action chunk.

Factor	Tested settings	Mean success (%)
Denoising groups (M)	1 / 3 / 5 / 10	95.5 / 97.3 / 97.1 / 97.4
Uncertainty margin (α)	0 / 1 / 3 / 5	96.3 / 96.9 / 97.3 / 94.8
Action context	Prefix / Suffix / Full chunk	95.9 / 96.3 / 97.3
Predictor sharing	Shared / Per-layer	96.4 / 97.3
Calibration trajectories	2 / 4 / 6	96.1 / 96.8 / 97.3

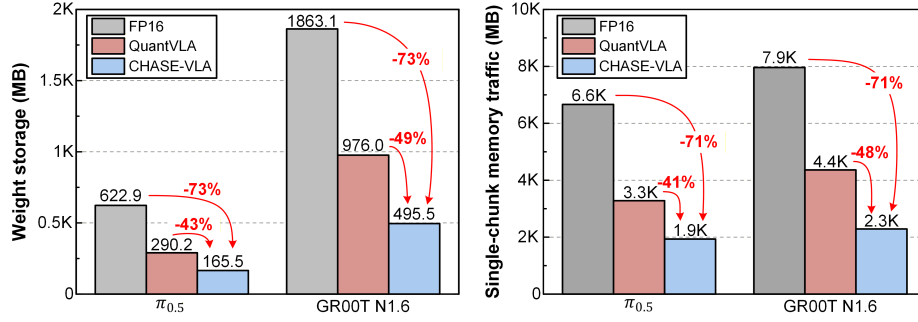


Fig. 4: Covered AE linear layer weight storage and single-chunk memory traffic on $\pi_{0.5}$ and GR00T N1.6, comparing FP16, QuantVLA, and CHASE-VLA.

4.3 Ablation Study

Table 4 evaluates five design choices on $\pi_{0.5}$ across the four LIBERO suites, with the default CHASE-VLA configurations marked in bold.

Three denoising step groups yield 97.3% success versus 95.5% with one group. Ten groups yield 97.4% with more predictor outputs, motivating the default $M = 3$. For the uncertainty margin, $\alpha = 3$ gives the highest observed success; increasing it to 5 broadens the scale range and yields 94.8%.

Full previous-chunk context yields 97.3%, versus 95.9% for the executed prefix and 96.3% for the unexecuted suffix. Per-layer prediction yields 97.3%, compared with 96.4% for shared prediction. Calibration with two, four, and six trajectories yields 96.1%, 96.8%, and 97.3%. The default uses two trajectories from each suite other than the evaluated suite.

4.4 Efficiency and Overhead Analysis

We compare storage and memory traffic against QuantVLA’s original MLP-only target, and measure AE latency on an NVIDIA A100 with batch size one.

Table 5: End-to-end action expert latency and speedup on LIBERO, measured on an NVIDIA A100 GPU with batch size one. CHASE-VLA applies W4A4 quantization to all MLP and attention linear projections in the action expert. Speedups are computed relative to the corresponding FP16 baseline.

Model	FP16	W4A8		CHASE-VLA (W4A4)	
	Latency (ms)	Latency (ms)	Speedup	Latency (ms)	Speedup
$\pi_{0.5}$	26.957	25.585	1.054×	21.069	1.279×
GR00T N1.6	43.003	41.889	1.027×	37.563	1.145×

Fig. 4 shows a 73.4% reduction in quantized AE linear weight storage on both models, versus 53.4% and 47.6% for QuantVLA on $\pi_{0.5}$ and GR00T N1.6. Single-chunk memory traffic decreases by 70.9% and 71.2% relative to FP16, reaching 1935.55 MB and 2288.65 MB, respectively.

Table 5 reports end-to-end AE latency. CHASE-VLA reduces AE latency by 21.8% on $\pi_{0.5}$ and 12.7% on GR00T N1.6 relative to FP16, and by 17.7% and 10.3% relative to W4A8, respectively. These latency gains are accompanied by a small predictor storage overhead: 5.76 MB on $\pi_{0.5}$ and 11.50 MB on GR00T N1.6, corresponding to 1.26% and 0.84% of the saved weight storage, respectively.

5 Conclusion

We introduced CHASE-VLA, a PTQ method that utilizes the generated action chunk as a causal signal for activation-scale estimation in diffusion-based VLA AEs. By applying action-conditioned scaling at the denoising step group level, CHASE-VLA enables W4A4 quantization of both MLP and attention projections in the repeated AE without requiring modifications to the pretrained policy. With W4A4 precision for AE linear layers, CHASE-VLA restores the FP16-level average LIBERO success on $\pi_{0.5}$ with 97.3%, and maintains 90.5% and 91.5% average success on CogACT and GR00T N1.6. It reduces the weight storage of the quantized AE linear layers by 73.4% on both $\pi_{0.5}$ and GR00T N1.6, and decreases single-chunk memory traffic by 70.9% and 71.2%, with predictor overheads of only 1.26% and 0.84% of the saved storage, respectively.

Acknowledgments. This work was supported by the National Research Foundation of Korea (NRF) grants funded by the Korea government (MSIT) (RS-2025-02264052 and RS-2025-09322969), and by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the MSIT (RS-2026-25531289, Development of Incrementally Adaptive On-device Learning Technology for New Environments).

References

1. Ashkboos, S., Mohtashami, A., Croci, M.L., Li, B., Cameron, P., Jaggi, M., Alistarh, D., Hoefler, T., Hensman, J.: QuaRot: Outlier-free 4-bit inference in rotated LLMs. *NeurIPS* **37**, 100213–100240 (2024)
2. Bjorck, J., Castañeda, F., Cherniadev, N., Da, X., Ding, R., Fan, L., Fang, Y., Fox, D., Hu, F., Huang, S., et al.: GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734* (2025)
3. Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M.R., Finn, C., Fusai, N., Galliker, M.Y., et al.: $\pi_{0.5}$: A vision-language-action model with open-world generalization. In: *CoRL*. vol. 305, pp. 17–40 (2025)
4. Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., et al.: π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164* (2024)
5. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Choromanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., et al.: RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818* (2023)
6. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Hsu, J., et al.: RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817* (2022)
7. Bu, Q., Yang, Y., Cai, J., Gao, S., Ren, G., Yao, M., Luo, P., Li, H.: Uni-VLA: Learning to act anywhere with task-centric latent actions. *arXiv preprint arXiv:2505.06111* (2025)
8. Chen, L., Meng, Y., Tang, C., Ma, X., Jiang, J., Wang, X., Wang, Z., Zhu, W.: Q-DiT: Accurate post-training quantization for diffusion transformers. In: *CVPR*. pp. 28306–28315 (2025)
9. Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., Song, S.: Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research* **44**(10-11), 1684–1704 (2025)
10. Frantar, E., Ashkboos, S., Hoefler, T., Alistarh, D.: GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022)
11. Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Kreiman, T., Xu, C., et al.: Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213* (2024)
12. Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., Kalenichenko, D.: Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: *CVPR*. pp. 2704–2713 (2018)
13. Kim, M.J., Finn, C., Liang, P.: Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645* (2025)
14. Kim, M.J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., et al.: OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246* (2024)
15. Li, Q., Liang, Y., Wang, Z., Luo, L., Chen, X., Liao, M., Wei, F., Deng, Y., Xu, S., Zhang, Y., et al.: CogACT: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650* (2024)
16. Lin, H., Xu, H., Wu, Y., Cui, J., Zhang, Y., Mou, L., Song, L., Sun, Z., Wei, Y.: DuQuant: Distributing outliers via dual transformation makes stronger quantized LLMs. *NeurIPS* **37**, 87766–87800 (2024)

17. Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.M., Wang, W.C., Xiao, G., Dang, X., Gan, C., Han, S.: AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In: *Proceedings of Machine Learning and Systems*. vol. 6, pp. 87–100 (2024)
18. Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., Stone, P.: LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *NeurIPS* **36**, 44776–44791 (2023)
19. Liu, S., Wu, L., Li, B., Tan, H., Chen, H., Wang, Z., Xu, K., Su, H., Zhu, J.: RDT-1B: A diffusion foundation model for bimanual manipulation. In: *ICLR*. pp. 29982–30009 (2025)
20. Nagel, M., Amjad, R.A., Van Baalen, M., Louizos, C., Blankevoort, T.: Up or down? adaptive rounding for post-training quantization. In: *ICML*. pp. 7197–7206 (2020)
21. Peebles, W., Xie, S.: Scalable diffusion models with transformers. In: *ICCV*. pp. 4195–4205 (2023)
22. Wang, H., Xiong, C., Wang, R., Chen, X.: BitVLA: 1-bit vision-language-action models for robotics manipulation. *arXiv preprint arXiv:2506.07530* (2025)
23. Wang, Y., Zhu, H., Liu, M., Yang, J., Fang, H.S., He, T.: VQ-VLA: Improving vision-language-action models via scaling vector-quantized action tokenizers. In: *ICCV*. pp. 11089–11099 (2025)
24. Wen, J., Zhu, Y., Li, J., Tang, Z., Shen, C., Feng, F.: DexVLA: Vision-language model with plug-in diffusion expert for general robot control. *arXiv preprint arXiv:2502.05855* (2025)
25. Wen, J., Zhu, Y., Li, J., Zhu, M., Tang, Z., Wu, K., Xu, Z., Liu, N., Cheng, R., Shen, C., et al.: TinyVLA: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters* **10**(4), 3988–3995 (2025)
26. Wen, J., Zhu, Y., Zhu, M., Tang, Z., Li, J., Zhou, Z., Liu, X., Shen, C., Peng, Y., Feng, F.: DiffusionVLA: Scaling robot foundation models via unified diffusion and autoregression. In: *ICML*. vol. 267, pp. 66558–66574 (2025)
27. Wu, J., Wang, H., Shang, Y., Shah, M., Yan, Y.: PTQ4DiT: Post-training quantization for diffusion transformers. *NeurIPS* **37**, 62732–62755 (2024)
28. Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., Han, S.: SmoothQuant: Accurate and efficient post-training quantization for large language models. In: *ICML*. pp. 38087–38099 (2023)
29. Xu, S., Wang, Y., Xia, C., Zhu, D., Huang, T., Xu, C.: VLA-Cache: Efficient vision-language-action manipulation via adaptive token caching. *NeurIPS* **38**, 164448–164473 (2025)
30. Xu, S., Wang, T., Li, F., Zhu, L., Shen, H.T.: DA-PTQ: Drift-aware post-training quantization for efficient vision-language-action models. *arXiv preprint arXiv:2604.11572* (2026)
31. Xu, Y., Yang, Y., Fan, Z., Liu, Y., Li, Y., Li, B., Zhang, Z.: QVLA: Not all channels are equal in vision-language-action model’s quantization. *arXiv preprint arXiv:2602.03782* (2026)
32. Yang, Y., Wang, Y., Wen, Z., Zhongwei, L., Zou, C., Zhang, Z., Wen, C., Zhang, L.: EfficientVLA: Training-free acceleration and compression for vision-language-action models. *NeurIPS* **38**, 40891–40914 (2025)
33. Zhang, J., Hsieh, Y., Wan, Z., Lin, H., Wang, X., Wang, Z., Lei, Y., Zhang, M.: QuantVLA: Scale-calibrated post-training quantization for vision-language-action models. In: *CVPR*. pp. 39539–39549 (2026)

34. Zhang, R., Dong, M., Zhang, Y., Heng, L., Chi, X., Dai, G., Du, L., Wang, D., Du, Y., Zhang, S.: MoLe-VLA: Dynamic layer-skipping vision language action model via mixture-of-layers for efficient robot manipulation. In: AAAI. pp. 18764–18772 (2026)
35. Zhao, T., Fang, T., Huang, H., Wan, R., Soedarmadji, W., Liu, E., Li, S., Lin, Z., Dai, G., Yan, S., Yang, H., Ning, X., Wang, Y.: ViDiT-Q: Efficient and accurate quantization of diffusion transformers for image and video generation. In: ICLR. pp. 65811–65841 (2025)
36. Zheng, Z., Cao, H., Tian, S., Chen, J., Li, M., Sun, X., Zou, H., Zhang, Z., Liu, X., Cao, D., et al.: DyQ-VLA: Temporal-dynamic-aware quantization for embodied vision-language-action models. arXiv preprint arXiv:2603.07904 (2026)

CHASE-VLA: Post-Training Quantization Framework for Vision-Language-Action Models with Chunk-Aware Scale Estimation (Supplementary Material)

Jin Hyun^{1,2}, Jung Gyu Min^{1,2}, Gyuhyun Jung^{1,2}, and Youngjoo Lee¹

¹ KAIST, Daejeon, South Korea

youngjoo@kaist.ac.kr

² POSTECH, Pohang, South Korea

{hyunj, mjg1104, jgh1020}@postech.ac.kr

1 Action Expert Architectures of VLA Models

Table 1 summarizes the action expert (AE) architectures and repeated AE linear layers of Vision-Language-Action (VLA) models.

Table 1: AE architectures and quantized AE linear layers.

Model	AE blocks	Denoising steps	Chunk ($H \times d_a$)	Target layers	Breakdown	
					MLP	Attn. proj.
$\pi_{0.5}$	18	10	50×7	126	54	72
CogACT	12	10	16×7	48	24	24
GR00T N1.6	32	4	16×7	192	64	128

The chunk column reports the action chunk shape $H \times d_a$ used for CHASE-VLA chunk conditioning, where H is the action horizon and d_a is the action dimension. The breakdown columns report the number of linear layers in each model’s implementation: CogACT uses fused attention projections, whereas $\pi_{0.5}$ and GR00T N1.6 utilize separate attention projections. For GR00T N1.6, the larger number of AE blocks and separate attention projections lead to a larger quantization target relative to $\pi_{0.5}$ and CogACT.

2 CHASE-VLA Predictor Architecture

CHASE-VLA uses a separate scale predictor for each quantized AE linear layer, since activation-scale statistics differ across the target linear layers. Each predictor is stored in 16-bit floating point (FP16) and maps the action context

Table 2: CHASE-VLA predictor architecture with a two-layer MLP.

Model	Predictors	d_{in}	d_{hid}	M	d_{out}
$\pi_{0.5}$	126	350	64	3	6
CogACT	48	112	256	3	6
GR00T N1.6	192	112	256	2	4

through a two-layer MLP with a ReLU activation between the two linear layers, producing

$$f_{\phi_l}(\text{norm}(z)) \in \mathbb{R}^{2M}, \quad (1)$$

where z denotes the flattened previous action chunk of the corresponding VLA model, and $\text{norm}(\cdot)$ denotes feature standardization. The outputs are organized as one log-scale adjustment and one log-variance estimate for each denoising step group. Table 2 summarizes the predictor count and the two-layer MLP dimensions used for each VLA model.

In Table 2, d_{in} denotes the flattened action-context dimension, and d_{hid} denotes the hidden width. M denotes the number of denoising step groups, and $d_{\text{out}} = 2M$ is the predictor output dimension. The model-dependent value of d_{in} follows the action chunk shape in Table 1: $\pi_{0.5}$ uses $H = 50$ with $d_a = 7$, while CogACT and GR00T N1.6 use $H = 16$ with $d_a = 7$.

3 CHASE-VLA Predictor Loss

CHASE-VLA fits its scale predictors as part of post-training calibration, with the pretrained VLA policy and AE weights kept frozen. Only the predictor parameters ϕ_l are optimized on the calibration queries. For each target linear layer l and denoising step group m , the calibration target is the base-10 log-ratio between the current-chunk scale and the group-specific initial scale:

$$y_{t_i, l, m}^* = \log_{10} \left(\epsilon + \frac{s_{t_i, l, m}^{\text{chunk}}}{s_{l, m}^{\text{init}}} \right). \quad (2)$$

In Eq. 2, t_i denotes the query time of the i -th collected calibration query, $m \in \{1, \dots, M\}$ indexes denoising step groups, and ϵ is a small numerical constant. In this target, $s_{l, m}^{\text{init}}$ denotes the calibration-derived group-specific base scale used when no previous action chunk is available, while $s_{t_i, l, m}^{\text{chunk}}$ denotes the scale measured from activations recorded during generation of the corresponding current chunk.

For each target linear layer l , the predictor takes the action context as input and outputs one log-scale adjustment and one log-variance estimate per denoising step group:

$$[\hat{y}_{t_i, l, 1:M}, \log \hat{\sigma}_{t_i, l, 1:M}^2] = f_{\phi_l}(\text{norm}(z_{t_i})), \quad z_{t_i} = \text{vec}(A_{t_i - \tau}). \quad (3)$$

The offset τ is the number of actions executed before the next policy query. The residual in log-scale space is

$$\Delta_{t_i,l,m} = y_{t_i,l,m}^* - \hat{y}_{t_i,l,m}. \quad (4)$$

The predictor objective combines a pinball term, a heteroscedastic Gaussian negative log-likelihood (NLL) term for the log-variance head, and a mean-squared parameter penalty. The pinball term is

$$\mathcal{L}_{\text{pin}} = \frac{1}{|\mathcal{D}_l|M} \sum_{i \in \mathcal{D}_l} \sum_{m=1}^M \max(\beta \Delta_{t_i,l,m}, (\beta - 1) \Delta_{t_i,l,m}). \quad (5)$$

In Eq. 5, $\mathcal{D}_l$ is the set of collected calibration queries for which layer l is observed, and β is the quantile level. Minimizing $\mathcal{L}_{\text{pin}}$ fits a high quantile of the log-ratio target rather than its conditional mean, so that the predicted adjustment reflects large activation ranges that are critical under low-bit activation quantization. The heteroscedastic NLL term is

$$\mathcal{L}_{\text{nll}} = \frac{1}{|\mathcal{D}_l|M} \sum_{i \in \mathcal{D}_l} \sum_{m=1}^M \left[\frac{1}{2} \exp(-\log \hat{\sigma}_{t_i,l,m}^2) \Delta_{t_i,l,m}^2 + \frac{1}{2} \log \hat{\sigma}_{t_i,l,m}^2 \right]. \quad (6)$$

This term supervises the log-variance head in log-scale space; the predicted standard deviation, scaled by α , provides the uncertainty margin used in the scale multiplier γ in Eq. 9. The mean-squared parameter penalty is

$$\mathcal{L}_{\text{reg}} = \sum_{p \in \phi_l} \text{mean}(p^2). \quad (7)$$

This penalty regularizes the lightweight predictor and mitigates overfitting to the calibration queries.

The final predictor objective is

$$\mathcal{L}_{\text{CHASE}} = \mathcal{L}_{\text{pin}} + \lambda \mathcal{L}_{\text{nll}} + \eta \mathcal{L}_{\text{reg}}. \quad (8)$$

In the evaluation, we set $\beta = 0.95$, $\lambda = 0.5$, and $\eta = 10^{-2}$.

At inference time, the predicted uncertainty is used only to form the scale multiplier applied to the group-specific initial scale:

$$\hat{\sigma}_{t,l,m} = \exp\left(\frac{1}{2} \log \hat{\sigma}_{t,l,m}^2\right), \quad \gamma_{t,l,m} = 10^{\hat{y}_{t,l,m} + \alpha \hat{\sigma}_{t,l,m}}, \quad (9)$$

where we set $\alpha = 3.0$.

4 CHASE-VLA Post-Training Quantization Details

4.1 Calibration Setup

In CHASE-VLA, post-training calibration estimates the quantization scales of the target AE linear layers and the activation-scale statistics used to fit the

predictors. Calibration is performed with the pretrained VLA policy and AE weights kept frozen throughout the procedure. For each LIBERO evaluation suite, we construct a calibration set by randomly sampling two trajectories from each of the other LIBERO suites. The evaluated suite is excluded from this sampling process, which prevents trajectory overlap between the calibration and evaluation sets. The set is used only to estimate activation-scale statistics and to fit the scale predictors.

4.2 Weight Quantization and Denoising Step Groups

The quantization configuration is fixed from calibration through inference. For the quantized AE linear layer weights, we apply symmetric per-row 4-bit quantization with the round-to-nearest method. The denoising steps are partitioned once for each AE, and the resulting M groups are reused throughout calibration and evaluation. For $\pi_{0.5}$ and CogACT, which use $K = 10$ denoising steps, the groups have sizes 3, 4, and 3 and correspond to step indices $\{0, 1, 2\}$, $\{3, 4, 5, 6\}$, and $\{7, 8, 9\}$. For GR00T N1.6, which uses $K = 4$ denoising steps, the groups have sizes 2 and 2 and correspond to step indices $\{0, 1\}$ and $\{2, 3\}$.

5 Confidence Intervals for LIBERO Success Rates

Table 3 reports 95% confidence intervals for the average LIBERO success rates presented in the main paper. Expanded QuantVLA applies W4A4 quantization to both MLP and attention projections in the AE, matching the quantization target of CHASE-VLA.

Table 3: 95% confidence intervals for average LIBERO success rates (%).

Method	$\pi_{0.5}$	CogACT	GR00T N1.6
FP16	[95.7, 98.1]	[91.2, 94.7]	[96.8, 98.8]
QuantVLA (expanded)	[93.6, 96.5]	[72.3, 78.2]	[45.9, 52.8]
Q-DiT	[92.6, 95.8]	[74.9, 80.6]	[20.1, 25.9]
PTQ4DiT	[91.6, 95.0]	[76.7, 82.3]	[17.4, 22.9]
CHASE-VLA	[95.9, 98.2]	[88.3, 92.3]	[89.4, 93.2]

On $\pi_{0.5}$, the CHASE-VLA interval overlaps the FP16 interval, and the point estimates remain close. On CogACT and GR00T N1.6, the CHASE-VLA intervals are above and do not overlap those of the compared W4A4 PTQ baselines.

6 Predictor-Free Scaling Baselines

Table 4 compares CHASE-VLA with three predictor-free scale-selection rules on $\pi_{0.5}$ LIBERO. Prev-query AbsMax reuses the previous query’s activation

maxima for each layer and denoising step group. AbsMax EMA smooths these maxima with an exponential moving average using $\rho = 0.9$. Motion lookup maps the previous action chunk’s energy and variation to calibrated scale tables.

Table 4: LIBERO success rates (%) for predictor-free baselines and CHASE-VLA on $\pi_{0.5}$.

Scale adjustment	Spatial	Object	Goal	Long	Avg.
Prev-query AbsMax	97.0	98.0	96.5	92.0	95.9
AbsMax EMA	97.5	98.5	95.5	92.5	96.0
Motion lookup	97.0	98.5	97.5	92.5	96.4
CHASE-VLA	97.0	99.5	98.0	94.5	97.3

CHASE-VLA exceeds the strongest predictor-free baseline, motion lookup, by 0.9 percentage points in average success.

7 SimplerEnv Evaluation

Table 5 evaluates GR00T N1.6 on SimplerEnv under Variant Aggregation (VA) and Visual Matching (VM). For the quantized methods, the VLM remains in FP16 while the AE QKVO projections and MLP layers use W4A4. CHASE-VLA differs from FP16 by +0.45, -2.67 , and -1.72 percentage points on Google Robot (VA), Google Robot (VM), and WidowX (VM), respectively.

Table 5: GR00T N1.6 success rates (%) on SimplerEnv. VA denotes Variant Aggregation and VM denotes Visual Matching.

Evaluation setup	FP16	QuantVLA	CHASE-VLA
Google Robot (VA)	46.54	29.47	46.99
Google Robot (VM)	56.00	43.67	53.33
WidowX (VM)	64.86	59.71	63.14